

Sam & Lauren wedding website: project brief

2026-09-17 · @Someone

Overview

A guest website for Sam and Lauren's weekend wedding at The Lodge Jamberoo, Friday 12 to Sunday 14 February 2027, built on Next.js and Sanity with 3D characters of the couple as the signature feature.

The site gives every guest the weekend plan, their room and their RSVP in one place, with a playful "Select Player" moment that makes it feel like us rather than a template.

Audience: up to about 50 guests, including families with young children. Most will visit on their phones.

Goals

- Collect RSVPs per day (Friday dinner, Saturday wedding, Sunday breakfast), plus dietary needs and kids' details.
- Explain the accommodation and the contribution ask clearly and without awkwardness.
- Give a clear, scannable schedule and practical info (getting there, kids, what's on site).
- Delight guests with the 3D characters and player pages.

Success criteria

- Every invited guest has RSVP'd by the deadline.
- The page is usable within 2.5 seconds on a mid-range phone over 4G, with the 3D loading after the content.
- Sam and Lauren can update copy, schedule times and models in Sanity without a redeploy.

Event details

The weekend runs Friday to Sunday at The Lodge Jamberoo, 406 Jamberoo Mountain Rd, Jamberoo NSW 2533, with guests staying on site ([venue](#)).

Schedule

Day	Event	Where	Time
Fri 12 Feb	Arrivals and check-in	The Lodge	TBC
Fri 12 Feb	Arrival dinner	TBC (options include the Champagne Garden, wedding terrace or Fin's Bar)	TBC
Sat 13 Feb	Ceremony	Tree Cathedral, the outdoor ceremony space by the river	TBC
Sat 13 Feb	Cocktail hour	Around the firepits and yard games	TBC
Sat 13 Feb	Reception	Wedding Hall	TBC
Sun 14 Feb	Recovery breakfast	Lulu's	TBC
Sun 14 Feb	Check-out	The Lodge	TBC

The venue also offers an optional guest yoga class, pool party add-on and spa treatments; none are confirmed for our weekend yet.

Accommodation

We're prepaying the onsite rooms and asking guests to contribute. The Lodge has 39 rooms across 9 acres of gardens, pine trees and river frontage ([rooms](#)).

Room	Beds	Sleeps	Notes
King Room	1 king, ensuite	2	Furnished balcony with pool view; 2 extra beds at a charge, cot free
Twin Double	2 doubles, ensuite	4	Balcony with pool view; suits a family with two kids or four friends
Family Room	King in one room, 2 singles and a lounge in the other	4 (plus 2 extra beds at a charge)	Spa bath; no direct pool access or view

On site

- **Lulu's:** the restaurant, with original art, ceiling murals and antique furniture. Lunch 12pm to 3pm, dinner 5pm to 10pm, last orders 8:30pm ([dine](#)).
- **Fin's Bar:** a dark, moody cocktail bar off Lulu's, open 12pm until late.
- **Champagne Garden and The Terrace:** outdoor and covered drinks spaces, 12pm to 8pm.
- **Facilities:** Palm Springs style pool with pool service from 12pm, Swedish sauna, outdoor hot tub, day spa, gym, games room, tennis court and yard games.

Kids

We're providing nannies for young children across the weekend. Times, age range and location are still to be confirmed.

Scope

Nine routes: a home page with the character select, two player pages, and six content pages.

```
flowchart TD
    H["Home<br/>Select Player"] --> S["/sam"]
    H --> L["/lauren"]
    H --> W["/weekend"]
    H --> ST["/stay"]
    H --> LO["/the-lodge"]
    H --> K["/kids"]
    H --> R["/rsvp"]
    H --> F["/faq"]
    S <--> L
```

Sam and Lauren's pages link to each other through a "Switch player" control.

Route	Purpose	Key content
/	First impression and navigation	3D couple idling, "SELECT PLAYER" prompt, names and dates in oversized type, nav
/sam , /lauren	Fun intro to each of us	Character dancing, big name heading, player card with playful stats, Switch player and Back controls
/weekend	The plan	Three-day schedule, dress code, what to bring

Route	Purpose	Key content
/stay	Rooms and contribution	Room types, how allocation works, contribution ask, check-in and check-out times
/the-lodge	What's on site	Lulu's, Fin's Bar, Champagne Garden, pool, spa, games, getting there
/kids	Reassure parents	Nanny details, times, ages, what to pack
/rsvp	Collect responses	Per-guest form with per-day attendance, dietary needs, room preference, kids, song request
/faq	Practical answers	Getting there, parking, weather, gifts, plus-ones

Out of scope for now: gift registry, photo uploads, guest-built avatars and a live schedule on the day. Each could be a later phase.

Design direction

Big typography, minimal layouts and a quiet video-game layer, deliberately avoiding the usual wedding look.

Principles

- Editorial and confident: oversized display type, generous white space, strong grid.
- Not wedding-y: no script or cursive fonts, florals, gold foil or blush pink.
- The game theme is original and understated. It shows up in the Select Player moment, player cards and a few UI details, not everywhere. No references to existing game brands, characters or logos.
- Mobile-first, with desktop layouts designed alongside.

Typography

- Display: a bold grotesk for headings, set very large (names, dates, day headings).
- Body: a clean, readable sans.
- UI labels: a monospace for stats, times, tags and game-style prompts.

Palette

- Off-white background and near-black text as the base.
- Deep pine green, drawn from the venue's pine trees and river setting.
- One bright accent reserved for interactive states (selection, hover, the RSVP button).

Signature interactions

- Home: "SELECT PLAYER" above the idling couple. Hover or tap highlights a character with an outline, name tag and selection bracket.
- Player pages: the character dances beside a monospace player card.
- RSVP: the submit button is a "Ready" or "Press start" moment.

Components: nav, buttons, room card, schedule item, player card, form fields and toggles, FAQ accordion, footer.

Claude Design deliverables

- Home, Sam's player page, The Weekend, Stay and RSVP, at mobile and desktop.
- The component set above, with type scale, colour tokens and spacing documented for handoff.
- The 3D areas shown as labelled placeholders ("3D canvas, React Three Fiber") using the Meshy renders.

3D characters

Both characters are modelled and rigged in Meshy; the remaining work is animation, export, compression and the React Three Fiber scene.

```
flowchart LR
  A["Model in Meshy<br/>done"] --> B["Remesh 30K tris<br/>done"]
  B --> C["Rig in Meshy<br/>done"]
  C --> D["Add animations"]
  D --> E["Export GLB"]
  E --> F["Compress<br/>gltf-transform"]
  F --> G["R3F scene"]
```

Mixamo is the free fallback for rigging and animation if Meshy's results deform badly.

Animations

Clip	Used on	Behaviour
Idle	Home, player pages	Default loop
Wave	Home	Plays once on hover or tap, then back to Idle
Dance	Player pages	Loops while the page is open
Cheer (optional)	RSVP success	Plays once after submitting

Use the same clip names for both characters so one component handles both. Check the shoulders and Lauren's open shirt on each clip, and drop any clip that deforms badly.

Asset targets

- Geometry: about 30K triangles per character (fallback 10K if older phones struggle).
- Textures: 2K maximum, WebP; drop to 1K if memory is tight.
- File size: under 3 to 5MB per character after meshopt compression.
- Both characters exported at the same scale so their relative heights are correct.
- Checked in [glTF report](#) for triangle count, texture sizes and clip names.

```
npx @glTF-transform/cli optimize sam-raw.glb sam.glb \
  --compress meshopt --texture-compress webp --texture-size 2048
```

Scene behaviour

- A reusable `<Character url clip position rotation />` component using drei's `useGLTF` and `useAnimations`, cloning scenes with `SkeletonUtils`, with 0.3 second crossfades between clips.
- Home: both characters side by side on Idle; Wave on hover or tap; selecting a character routes to their page.
- Player pages: the selected character large and dancing, gentle orbit only (no pan or zoom, limited vertical angle).
- Lighting: a drei `Environment` preset plus `ContactShadows`; no real-time shadows.

Fallbacks

- `prefers-reduced-motion`: Idle only, no dance or autoplaying clips.
- No WebGL or a very slow device: a static render of each character in place of the canvas.
- While loading: a lightweight placeholder, so the page content is never blocked.

Development

Next.js App Router with Sanity as the CMS, built with Claude Code and hosted on Vercel.

Stack

- Next.js (App Router, TypeScript), Tailwind or CSS modules to match the design tokens.
- Sanity for all content, including schedule, rooms, player cards, FAQ and 3D model files.

- 3D: `three`, `@react-three/fiber`, `@react-three/drei`, `three-stdlib`. No other 3D dependencies.
- The 3D canvas loaded with `next/dynamic ( ssr: false )` inside `Suspense`, with models preloaded via `useGLTF.preload`.

Sanity schemas (draft)

These are a starting point and will change once the Claude Design screens are final. Fields should follow the designs, not the other way round.

Document	Key fields
<code>siteSettings</code> (singleton)	Names, dates, venue, RSVP deadline, hero copy, contribution settings and copy
<code>player</code>	Name, slug, order, GLB model, clip names, fallback image, tagline, configurable player card stats , bio
<code>event</code>	Title, day, start and end, location (reference), dress code, description, show on site
<code>venueSpot</code>	Name, slug, category (ceremony, reception, dining, bar, facility), description, hours, image
<code>roomType</code>	Name, beds, sleeps, notes, image, order
<code>faq</code>	Question, answer, order
<code>rsvp</code>	Guests (name, days attending, dietary needs, child and age), email, room preference, song request, message, submitted at

Player card stats are an array, so each stat is added, removed, renamed and reordered in the Studio with no code change. Each stat is either **text** ("Special move: the worm") or a **meter** ("Dance skill", 0 to 10) that the design can render as a game-style bar.

```
// schemas/player.ts
import {defineArrayMember, defineField, defineType} from 'sanity'

export const player = defineType({
  name: 'player',
  title: 'Player',
  type: 'document',
  fields: [
    defineField({name: 'name', type: 'string', validation: (r) =>
      r.required()}),
```

```

    defineField({name: 'slug', type: 'slug', options: {source: 'name'},
validation: (r) => r.required()}),
    defineField({name: 'order', type: 'number', description: 'Position on the
Select Player screen'}),
    defineField({name: 'model', title: 'Model (GLB)', type: 'file', options:
{accept: '.glb'}}),
    defineField({
      name: 'clips',
      title: 'Animation clips',
      type: 'object',
      description: 'Must match the clip names inside the GLB',
      fields: [
        defineField({name: 'idle', type: 'string', initialValue: 'Idle'}),
        defineField({name: 'hover', title: 'Hover or tap', type: 'string',
initialValue: 'Wave'}),
        defineField({name: 'feature', title: 'Player page', type: 'string',
initialValue: 'Dance'}),
      ],
    }),
    defineField({name: 'fallbackImage', type: 'image', description: 'Shown
when 3D is unavailable'}),
    defineField({name: 'tagline', type: 'string'}),
    defineField({
      name: 'stats',
      title: 'Player card stats',
      type: 'array',
      description: 'Add, remove, rename and reorder freely',
      of: [
        defineArrayMember({
          name: 'stat',
          type: 'object',
          fields: [
            defineField({name: 'label', type: 'string', validation: (r) =>
r.required()}),
            defineField({
              name: 'kind',
              type: 'string',
              options: {list: ['text', 'meter'], layout: 'radio'},
              initialValue: 'text',
            }),
            defineField({
              name: 'value',
              type: 'string',
              hidden: ({parent}) => parent?.kind === 'meter',
            }),
            defineField({

```



```

        name: 'level',
        type: 'number',
        description: '0 to 10',
        validation: (r) => r.min(0).max(10),
        hidden: ({parent}) => parent?.kind !== 'meter',
      }),
    ],
    preview: {select: {title: 'label', subtitle: 'value'}}},
  }),
],
}),
defineField({name: 'bio', type: 'array', of: [defineArrayMember({type:
'block'})]})),
],
})

```

```

// schemas/siteSettings.ts
export const siteSettings = defineType({
  name: 'siteSettings',
  title: 'Site settings',
  type: 'document',
  fields: [
    defineField({name: 'title', type: 'string', initialValue: 'Sam &
Lauren'})),
    defineField({name: 'startDate', type: 'date'})),
    defineField({name: 'endDate', type: 'date'})),
    defineField({
      name: 'venue',
      type: 'object',
      fields: [
        defineField({name: 'name', type: 'string'})),
        defineField({name: 'address', type: 'text', rows: 2}),
        defineField({name: 'mapUrl', type: 'url'})),
      ],
    }),
    defineField({name: 'selectPlayerLabel', type: 'string', initialValue:
'SELECT PLAYER'})),
    defineField({name: 'rsvpDeadline', type: 'date'})),
    defineField({
      name: 'contribution',
      type: 'object',
      fields: [
        defineField({name: 'showAmount', type: 'boolean', initialValue:
false}),
        defineField({
          name: 'amountPerNight',

```

```

        type: 'number',
        hidden: ({parent}) => !parent?.showAmount,
      }),
      defineField({name: 'copy', type: 'array', of:
[defineArrayMember({type: 'block'})]}),
        defineField({name: 'paymentDetails', type: 'text', description:
'Shown after RSVP only'}),
      ],
    })),
  ],
})

```

```

// schemas/event.ts
export const event = defineType({
  name: 'event',
  type: 'document',
  fields: [
    defineField({name: 'title', type: 'string', validation: (r) =>
r.required()}),
    defineField({
      name: 'day',
      type: 'string',
      options: {list: ['friday', 'saturday', 'sunday'], layout: 'radio'},
    }),
    defineField({name: 'start', type: 'datetime'}),
    defineField({name: 'end', type: 'datetime'}),
    defineField({name: 'location', type: 'reference', to: [{type:
'venueSpot'}]}),
    defineField({name: 'dressCode', type: 'string'}),
    defineField({name: 'description', type: 'array', of:
[defineArrayMember({type: 'block'})]}),
    defineField({name: 'showOnSite', type: 'boolean', initialValue: true}),
  ],
  orderings: [{title: 'Start time', name: 'start', by: [{field: 'start',
direction: 'asc'}]}],
})

```

```

// schemas/venueSpot.ts
export const venueSpot = defineType({
  name: 'venueSpot',
  title: 'Venue spot',
  type: 'document',
  fields: [
    defineField({name: 'name', type: 'string', validation: (r) =>
r.required()}),
    defineField({name: 'slug', type: 'slug', options: {source: 'name'}}),

```

```

    defineField({
      name: 'category',
      type: 'string',
      options: {list: ['ceremony', 'reception', 'dining', 'bar',
'facility']},
    }),
    defineField({name: 'description', type: 'text', rows: 3}),
    defineField({name: 'hours', type: 'string'}),
    defineField({name: 'image', type: 'image', options: {hotspot: true}}),
  ],
})

// schemas/roomType.ts
export const roomType = defineType({
  name: 'roomType',
  title: 'Room type',
  type: 'document',
  fields: [
    defineField({name: 'name', type: 'string', validation: (r) =>
r.required()}),
    defineField({name: 'beds', type: 'string'}),
    defineField({name: 'sleeps', type: 'number'}),
    defineField({name: 'notes', type: 'text', rows: 3}),
    defineField({name: 'image', type: 'image', options: {hotspot: true}}),
    defineField({name: 'order', type: 'number'}),
  ],
})

// schemas/faq.ts
export const faq = defineType({
  name: 'faq',
  title: 'FAQ',
  type: 'document',
  fields: [
    defineField({name: 'question', type: 'string', validation: (r) =>
r.required()}),
    defineField({name: 'answer', type: 'array', of: [defineArrayMember({type:
'block'})]}),
    defineField({name: 'order', type: 'number'}),
  ],
})

// schemas/rsvp.ts (written by the server action, read-only in the Studio)
export const rsvp = defineType({
  name: 'rsvp',
  title: 'RSVP',

```

```

type: 'document',
readOnly: true,
fields: [
  defineField({
    name: 'guests',
    type: 'array',
    of: [
      defineArrayMember({
        name: 'guest',
        type: 'object',
        fields: [
          defineField({name: 'name', type: 'string'}),
          defineField({
            name: 'attending',
            type: 'array',
            of: [defineArrayMember({type: 'string'})],
            options: {list: ['friday', 'saturday', 'sunday']},
          }),
          defineField({name: 'dietary', type: 'string'}),
          defineField({name: 'isChild', type: 'boolean', initialValue:
false}),
          defineField({name: 'age', type: 'number', hidden: ({parent}) =>
!parent?.isChild}),
        ],
      }),
    ],
  }),
  defineField({name: 'email', type: 'email'}),
  defineField({name: 'roomPreference', type: 'reference', to: [{type:
'roomType'}]}),
  defineField({name: 'songRequest', type: 'string'}),
  defineField({name: 'message', type: 'text', rows: 3}),
  defineField({name: 'submittedAt', type: 'datetime'}),
],
preview: {select: {title: 'guests.0.name', subtitle: 'submittedAt'}},
})

```

The other files import `defineType`, `defineField` and `defineArrayMember` from `sanity`, as in `player.ts`. `siteSettings` should be pinned as a singleton in the Studio structure.

RSVP flow

- The form posts to a Next.js server action that validates the input and writes an `rsvp` document with a server-only Sanity write token.

- Spam protection with a honeypot field plus rate limiting, or an invite code per household.
- A confirmation screen (with the Cheer animation), and optionally a confirmation email.
- Responses are reviewed in a custom Sanity Studio view or exported to CSV.

Privacy

- `noindex` on every page so the site stays out of search results.
- Optional site-wide password or invite code, given it lists a location, dates and kids' details.

Performance budget

- Largest Contentful Paint under 2.5 seconds on a mid-range phone over 4G.
- 3D never blocks content: text and navigation render first.
- Canvas `dpr={[1, 2]}` so high-density phones don't render at 3x.
- Next.js `Image` for all photos; fonts self-hosted with `next/font`.

Accessibility

- Character selection works with keyboard and screen readers (real links with labels, not just canvas clicks).
- Reduced-motion support as described in the 3D section.
- Colour contrast at WCAG AA for text and the accent colour.

Testing and launch

- Test on at least one iPhone and one older Android over 4G.
- Vercel preview deployments for review; production on a custom domain.

Content and copy

The tone is warm, relaxed and a little playful, matching the design rather than formal wedding language.

Accommodation contribution

Two options, depending on whether we name an amount on the site:

1. **With an amount:** "We've booked and paid for the rooms upfront so no one has to worry about logistics. If you're able to, we'd be grateful for a contribution of \$[amount] per room, per night. We'll share the details with your RSVP, and if that's tricky for any reason, just let us know."
2. **Softer, no amount:** "We've taken care of the rooms so everyone can stay together. If you'd like to contribute towards yours, we'll send the details with your RSVP confirmation."

Player cards

Stats are fully configurable in Sanity, so these are starting suggestions only. Each can be a text stat or a 0 to 10 meter, and the two cards don't need identical stats:

- Home town (text)
- Special move (text)
- Weakness (text)
- Favourite drink at Fin's (text)
- How we met, from each person's point of view (text)
- Dance skill (meter)
- Karaoke confidence (meter)

Content to gather

- ☐ Event times for all three days
- ☐ Arrival dinner location and format
- ☐ Dress code for each event
- ☐ Check-in and check-out times
- ☐ Contribution amount and payment method
- ☐ Nanny times, age range and what parents should pack
- ☐ Player card answers for Sam and Lauren
- ☐ Getting there, parking and transport notes
- ☐ FAQ answers (gifts, plus-ones, weather plan)
- ☐ Photos: venue (with permission) and of us

Timeline

The site is designed and built this weekend, 19 to 20 September, then launches with the invitations in mid-November, about three months before the wedding.

Milestone	Target	Notes
Design in Claude Design	2026-09-19	Saturday morning: home, player page, Weekend, Stay, RSVP at mobile and desktop
3D animations exported and compressed	2026-09-19	Saturday: Idle, Wave, Dance (plus Cheer if it works)
Core build and Sanity schemas	2026-09-20	Sunday: all routes, 3D scene, RSVP flow
Content entered	2026-09-20	Sunday: everything we have now; gaps filled in Sanity as answers come in
Device testing and fixes	2026-09-20	Sunday evening: iPhone and older Android over 4G
Launch with invitations	2026-11-13	Site live, password or invite codes sent
RSVP deadline	2026-12-11	Before the Christmas break, so numbers and rooms can be confirmed
Final details update	2027-01-29	Times, nanny details, weather plan
Wedding weekend	2027-02-12	Friday 12 to Sunday 14 February

Open questions

- ☐ Confirm the weekend is 12 to 14 February 2027.
- ☐ Name a contribution amount on the site, or share it privately after RSVP?
- ☐ How do guests pay the contribution (bank transfer, PayID, a payment link)?
- ☐ Who allocates rooms: us upfront, or guests choosing a room type in the RSVP?
- ☐ Password-protect the site, or use per-household invite codes?
- ☐ Where and how is the Friday arrival dinner run?
- ☐ Add venue extras (guest yoga, pool party, spa treatments) to the schedule?
- ☐ Does Meshy's rigging hold up for the Dance clip, or do we switch to Mixamo?
- ☐ Domain name for the site.
- ☐ Send a confirmation email after RSVP?

Sources

- [The Lodge Jamberoo: Weddings](#)

- [The Lodge Jamberoo: Rooms](#)
- [The Lodge Jamberoo: Dine \(Lulu's and Fin's Bar\)](#)